

19. Frontend Module

19.1 Composition

The client is a single-page React 19 application written in TypeScript and bundled by Vite. Styling uses TailwindCSS with a small set of components built over Radix UI primitives — select, progress, label, separator, dialog, tooltip — which supply keyboard handling and ARIA semantics that a hand-rolled dropdown would not. Charts are drawn with Recharts. Forms are managed by React Hook Form and validated by Zod. Routing uses React Router.

Five routes are declared in `App.tsx` :

Path	View
/	Landing
/assessment	Four-step input form
/results/:id	Prediction breakdown
/history	Past assessments
/analytics	Aggregate dashboard

19.2 Separation from Business Logic

The frontend computes nothing. It holds no thresholds, no scoring weights, no risk bands and no model. Every number it renders arrives from the service, and its only substantive responsibility beyond presentation is input validation.

That validation duplicates the backend's, deliberately. The Zod schema in `Assessment.tsx` enumerates the same literal values as the Pydantic `Literal` types, so a user selecting an invalid option is told immediately rather than after a round trip. The duplication is a usability affordance, not a source of truth; the backend rejects independently, and a divergence between the two is a defect in the frontend rather than a relaxation of the rule.

The two did diverge. An earlier build of the form presented `occupation` and `location` as free-text inputs, and offered dropdown options — `Bachelor's`, `Master's`, `Upper-Middle`, `Full-time`, `Daily` for investment frequency, `Health Only` for insurance — that appear nowhere in the training vocabulary. Every submission carrying one of these was rejected with HTTP 422. The fields are now `Select` components restricted to the ten permitted literal sets given in Section 13.3, and the Zod schema uses `z.enum` rather than `z.string`, so an invalid value cannot survive the type checker.

19.3 Type Safety Against the Contract

All request and response shapes are declared in `src/types/index.ts`, and every call passes through `src/api/assessments.ts`. No page component constructs a URL or touches Axios. This confines the effect of an interface change to two files, and `npm run build` runs `tsc -b` before Vite, so a type that no longer

matches its usage fails the build rather than the runtime.

The API layer also absorbs the envelope inconsistency described in Section 18.3. `postPredict` returns `response.data.data`; `getAnalytics` returns `response.data`. Components receive the inner payload in both cases and are unaware that the two endpoints disagree about shape.

19.4 Views

Landing presents the system and routes to the assessment.

Assessment is a four-step wizard over the twenty-six inputs, grouped as demographics, financial profile, lifestyle and review. Each step is validated with `trigger()` on its own field subset before the user may advance, so errors surface on the step that produced them rather than at submission.

Results renders the prediction. A Recharts `RadialBarChart` displays the confidence of the predicted class, coloured green, amber or red by risk band. A horizontal bar chart shows the ten SHAP attributions. The component colours each bar blue for a positive value and red for a negative one, and captions the chart accordingly, but no bar is ever red: the service returns absolute magnitudes, as Section 21.4 explains. The legend describes a distinction the data does not carry. Three numeric cards show the decision, behavioural composite and discipline scores. Detected behavioural biases appear as badges, and the rule-generated recommendations as an ordered list.

History lists past assessments, each row linking to its results.

Analytics fetches `/analytics` and draws the risk and investment distributions as pie charts, the mean scores as a bar chart, and the per-day assessment count as a line chart.

Dark mode is provided by a `ThemeContext` that reads the system preference on first load and persists an override to `localStorage`.

19.5 Persistence of Submitted Assessments

The form submits to `POST /assessment`, receives an `assessment_id`, caches the payload in `sessionStorage` under that identifier for an immediate render, and navigates to `/results/{id}`. If a later visit to that route finds no cached entry — a reloaded tab, a shared link, a different browser — the page refetches through `GET /assessment/{id}`.

This was not the original behaviour, and the correction is worth recording because the failure was invisible to every test in the suite.

The form previously called `postPredict`, the endpoint that computes a prediction and deliberately does not store it. `postAssessment` existed in `src/api/assessments.ts`, was correctly typed, and was called from nowhere. The page synthesised an identifier from `Date.now()` and used it as though it were a database key.

Four consequences followed, each of which would have presented as an unrelated bug. Nothing a user entered through the interface reached the database, so the `assessments`, `predictions` and `behavioral_scores` tables held only rows created by direct `curl` calls during testing. The history view queried the database, found nothing the user had submitted, and silently fell back to a `localStorage` list

populated by an explicit "Save to History" button — so the history a user saw was a browser artefact rather than a record. The results route worked only from `sessionStorage`; a reload passed a millisecond timestamp to `GET /assessment/{id}` and received HTTP 404, which meant results were neither shareable nor durable. And the analytics dashboard aggregated a table the interface never wrote to.

The defect survived because the backend was correct. Its integration tests exercised `POST /assessment` directly and passed. The frontend compiled, and TypeScript had no opinion about which of two correctly-typed functions the page chose to call. Nothing in the system compared the endpoint the interface used against the endpoint the objectives required, because no test spanned both.

The repair changed two lines: the import, and the call site. Verification exercised the corrected path end to end against a fresh database. Submitting an assessment returned the UUID `5d0b172f-...`; `GET /assessment/{id}` resolved it; `GET /assessments` reported a total of one and listed that identifier; `GET /analytics` counted it. Inspecting the store afterwards showed one `assessments` row with `status = "completed"`, one `predictions` row carrying its SHAP attributions, and three `behavioral_scores` rows for the composite, discipline and decision scores. The `users` table remained empty, as expected in a system without authentication.

The end-to-end flow stated in the objectives of Section 7 — a user submits, the model predicts, the prediction is stored, the dashboard renders the history — is exercised by the running application.

19.6 Build and Serving

`npm run build` type-checks with `tsc -b` and then bundles with Vite. The production bundle is 922.99 kB of JavaScript, 279.68 kB after gzip, alongside 20.79 kB of CSS. The JavaScript chunk exceeds Vite's default 500 kB warning threshold, and the bulk of it is Recharts together with the D3 modules it depends on. Splitting the analytics and results routes behind `React.lazy` would bring the initial chunk under the threshold; this has not been done.

In deployment the assets are served by nginx from a multi-stage image, with `try_files $uri $uri/ /index.html` so that a reload on `/results/123` returns the application rather than a 404, and an `/api/` proxy to the backend service.

Screenshot 19.1 — *Landing page*, light and dark mode side by side, demonstrating the theme toggle described in Section 19.4.

Screenshot 19.2 — *Results page*, full view showing the radial confidence gauge, the three score cards, the SHAP bar chart with both positive and negative attributions visible, and the recommendations list. This is the single most important screenshot in the report. Place at the start of Section 19.4.

Screenshot 19.3 — *Analytics dashboard*. Submit several assessments through the form before capturing, so the distributions and the per-day line chart contain more than a single point.

Screenshot 19.4 — *Assessment form on a mobile viewport (375 px)*, demonstrating the responsive layout. Place in Section 19.4.

Figure 19.1 — *Component and data-flow diagram of the frontend*: App → route components → `api/assessments.ts` → Axios client → backend. Show `Assessment` calling `postAssessment` and `Results` falling back to `getAssessment` when the `sessionStorage` cache misses, since that pair is the subject of Section 19.5.